

COS 214 Practical 5

Team details:

Jordan Reddy - u25029666

Nasiha Osman - u25110188

Lwandiso George - u25405064

Github repo link: <https://github.com/jordanreddy10-cpu/COS-214-Practical-5>

Collaboration review:

We divided the work by pattern ownership so that each member had a clear area, while keeping the design decisions joint. We drafted the class diagram together before coding and agreed on the interfaces between our parts early.

Integration was the hardest part, because State and Strategy depend on the Mediator and Command classes. Running Valgrind on the combined build exposed a use-after-free in the Mediator, and testing our scenarios showed that the State pattern was not being exercised in Scenario 1. Both were fixed after we reviewed the code together. Before submission we held a walkthrough so every member could trace a command from invoker to receiver and explain each pattern's role.

Task 1: Analysis and Design

System Description:

CampusGuard is an emergency-response coordination platform designed for a large university campus. During a campus incident, multiple operational units (such as security teams, medical responders, facilities staff, and access-control systems) need to coordinate quickly and effectively.

Some of these systems are modern, while others rely on legacy or externally managed interfaces. The goal of this project is to build a cohesive C++11 application that manages the lifecycle of an incident from reporting to resolution, without tightly coupling every component to one another.

Participants:

Adapter:

- Target - Communicator
- Adapter - Adapter
- Adaptee - LegacyRadio

Command:

- Invoker - Colleague
- Command - Command
- Receiver - Colleague
- Concrete Commands - Operational (Dispatch, Notify), Access (Lock, Unlock, Restrict)

Mediator:

- Mediator - Mediator
- Concrete Mediator - Broadcaster
- Colleague - Colleague
- Concrete Colleagues - CampusSecurity, Medical, Staff

State:

- Context - Broadcaster
- State - IncidentState
- Concrete States - ReportedState, DispatchedState, InProgressState, SolvedState

Strategy:

- Context - Broadcaster
- Strategy - ResponseStrategy
- Concrete Strategy - MedStrategy, SecurityStrategy

Justifications:

We chose state and strategy so we could determine the level at which an incident was being dealt with and once we found an issue the method chosen to go about solving that issue.

State: We decided to use state to simulate how an actual emergency happens and is resolved. It does not instantly be created and then solved but rather goes through different

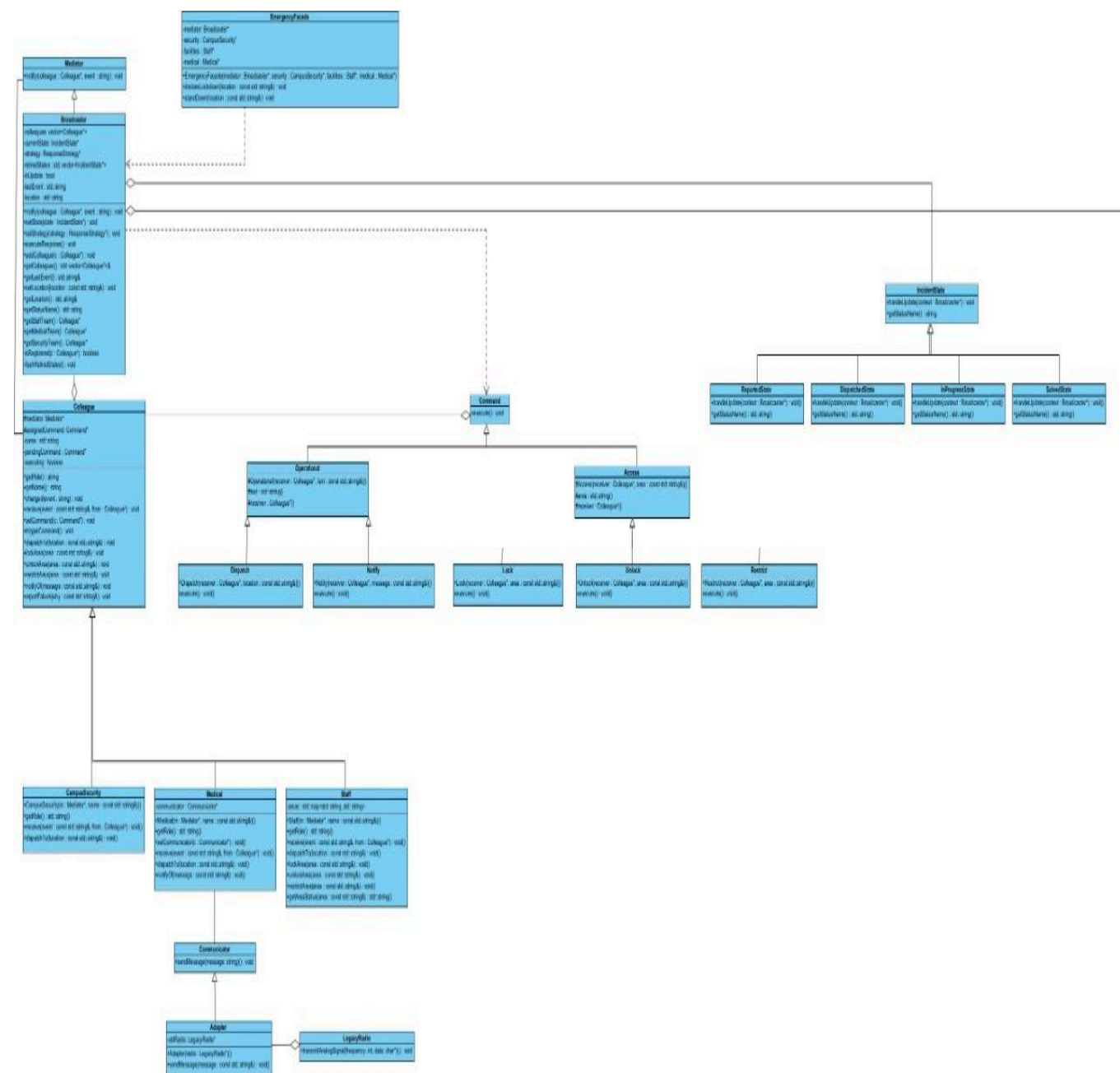
levels before being solved. This eliminates the need to use boolean values like `isResolved` when creating objects to represent an emergency.

Strategy: Because in real life you cannot apply the same step to solve every problem, we decided to simulate that in our system. This mainly takes the form of how the `CampusGuard` deals with medical and security based issues.

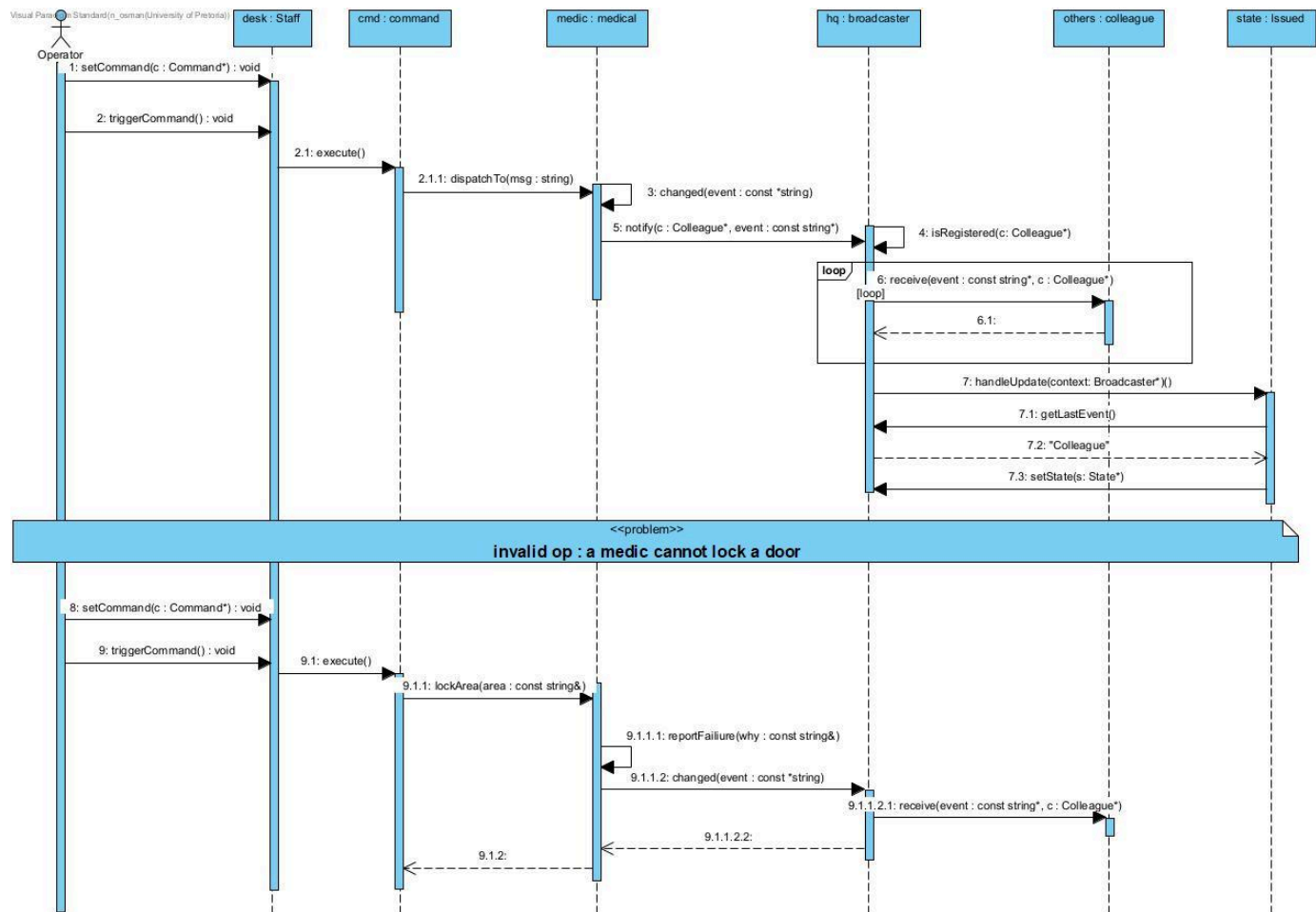
Ownership decisions:

The Concrete Mediator (`Broadcaster`) which also acts as the context for the State and Strategy patterns owns `IncidentState` and `ResponseStrategy`. When either `setState` or `setStrategy` is called `Broadcaster` takes ownership and then is responsible for deleting them. `Broadcaster` however does not own its colleagues. This is in line with the Mediator design pattern and ensures the colleagues have no way of knowing about each other. A colleague does however own its own Command object, this was so when a colleague were to be deleted the commands that were associated with it would also be deleted. Additionally the Medical Colleague does not own the `Communicator` class.

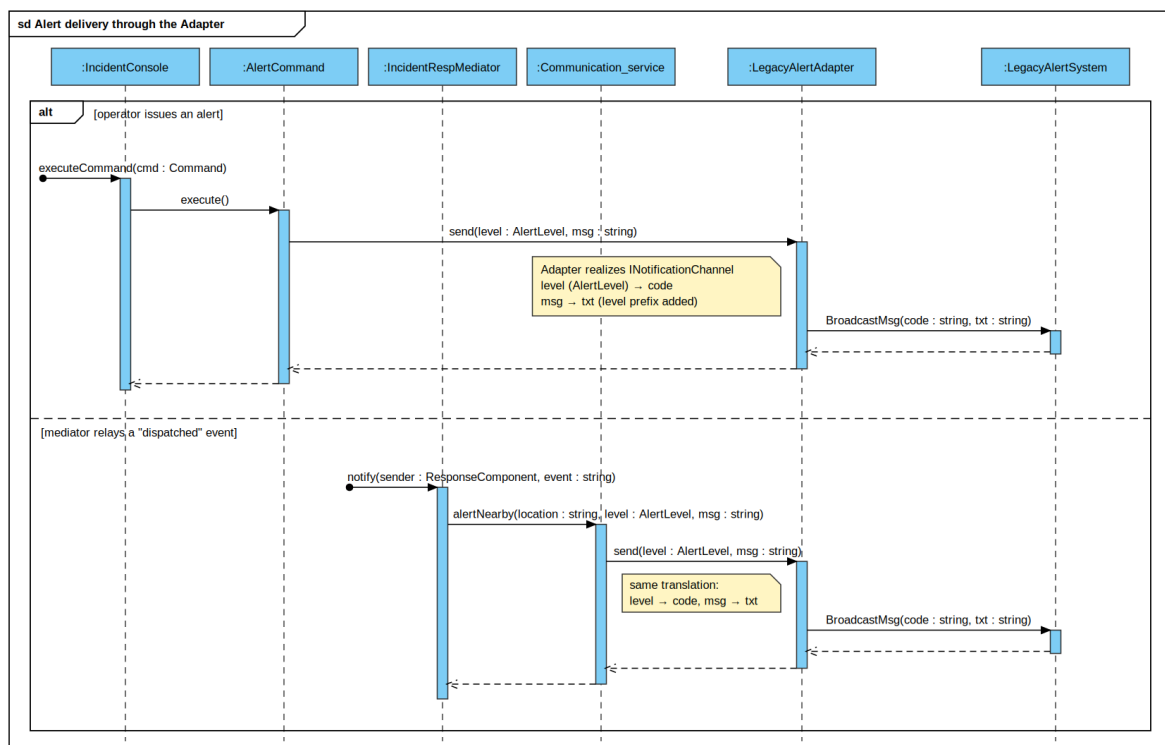
UML Diagram:



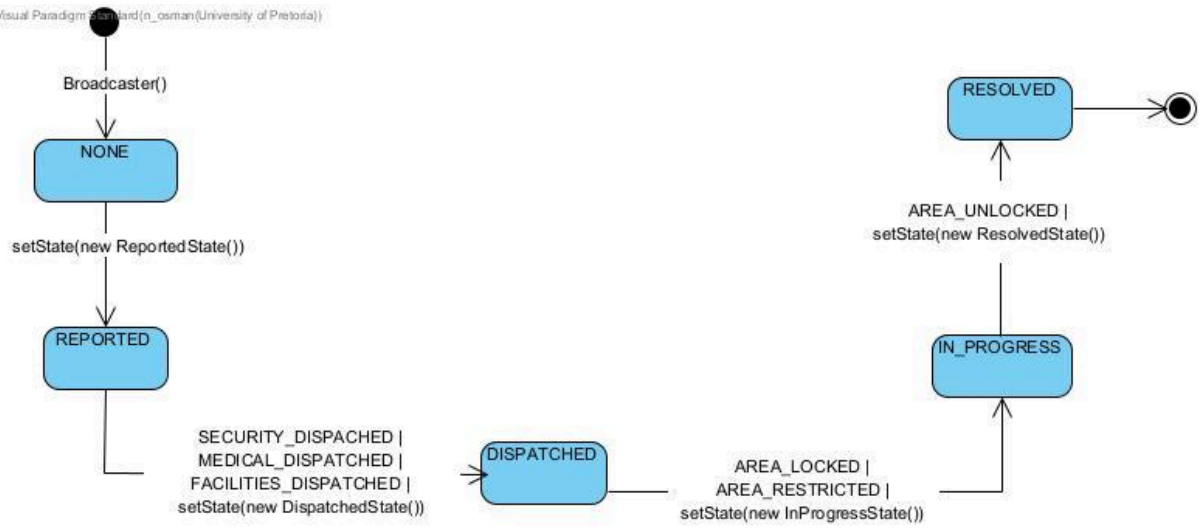
Sequence diagram depicting runtime interaction of command and mediator



Adapter sequence diagram:



State diagram as the 3rd required diagram



Task 5: Valgrind and GDB investigation

Final heap summary of program

```
==6497==
==6497== HEAP SUMMARY:
==6497==   in use at exit: 0 bytes in 0 blocks
==6497==   total heap usage: 56 allocs, 56 frees, 76,709 bytes allocated
==6497==
==6497== All heap blocks were freed -- no leaks are possible
==6497==
==6497== For lists of detected and suppressed errors, rerun with: -s
==6497== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

We encountered a segmentation fault during the implementation of our main.cpp. This issue was further investigated as follows:

```
[Notify] -> Med1: Lockdown in effect at Science Building - stand by.
sending message through adapter
older radio has freq: 155msg: [Med1] Lockdown in effect at Science Building - stand by.
[Mediator] Med1 reported NOTICE_SENT -> relaying to the other colleagues
[Mediator] ERROR: no response strategy selected, nothing to execute
=== [Facade] Lockdown workflow complete. Incident status: NONE ===

--- Step 2: Med1 goes off duty (deleted) but is still registered ---

--- Step 3: a new event is reported; mediator relays it to every colleague ---
[Mediator] Security1 reported SECURITY_DISPATCHED -> relaying to the other colleagues
[staff1] reacting to Security1: standing by to secure the area
Segmentation fault (core dumped)
```

Segmentation fault occurred during testing where staff1 reacts to SECURITY_DISPATCHED, at the moment the mediator reaches Med1.

```
==9979== LEAK SUMMARY:
==9979==   definitely lost: 0 bytes in 0 blocks
==9979==   indirectly lost: 0 bytes in 0 blocks
==9979==   possibly lost: 0 bytes in 0 blocks
==9979==   still reachable: 75,486 bytes in 18 blocks
==9979==   suppressed: 0 bytes in 0 blocks
==9979== Reachable blocks (those to which a pointer was found) are not shown.
==9979== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==9979==
==9979== For lists of detected and suppressed errors, rerun with: -s
==9979== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
Aborted (core dumped)
```

Upon further investigation, it was found that the error stemmed from an invalid read.

```

Breakpoint 1, main () at bug/bug_main.cpp:38
38         delete medical;
$1 = (Medical *) 0x5555557c460

--- Step 3: a new event is reported; mediator relays it to every colleague ---
[Mediator] Security1 reported SECURITY_DISPATCHED -> relaying to the other colleagues
[staff1] reacting to Security1: standing by to secure the area

Program received signal SIGSEGV, Segmentation fault.
0x000055555558309 in Broadcaster::notify (this=0x5555557c2b0, sender=0x5555557c390, event="SECURITY_DISPATCHED") at Broadcaster.cpp:46
46         snapshot[i]->receive(event, sender);
#0  0x000055555558309 in Broadcaster::notify (this=0x5555557c2b0, sender=0x5555557c390, event="SECURITY_DISPATCHED") at Broadcaster.cpp:46
#1  0x000055555555beb1 in Colleague::changed (this=0x5555557c390, event="SECURITY_DISPATCHED") at Colleague.cpp:20
#2  0x0000555555561ab1 in main () at bug/bug_main.cpp:42
#0  0x000055555558309 in Broadcaster::notify (this=0x5555557c2b0, sender=0x5555557c390, event="SECURITY_DISPATCHED") at Broadcaster.cpp:46
46         snapshot[i]->receive(event, sender);
i = 2
snapshot = std::vector of length 3, capacity 3 = {0x5555557c390, 0x5555557c3e0, 0x5555557c460}
n_osman@DESKTOP-B590PDE:/mnt/c/Users/n_osman/Downloads/COS-214-Practical-5-main1/COS-214-Practical-5-main$

```

A breakpoint was created to further investigate.

It was found that the crash is at Broadcaster.cpp:46 with $i = 2$. The same address occurred, so the deleted object is still in the mediator's list.

Task 6: System Integration, Containerization, and Memory Verification

- Pattern Integration: Successfully integrated and executed the complete Campus Guard incident management system across two primary scenarios, validating the interoperability of the Facade, Mediator, Command, Strategy, Adapter, and State design patterns.
- Environment Containerization: Authored Docker configuration files to create a reproducible, Ubuntu-based C++ build environment pre-configured with `g++`, `make`, `gdb`, and `valgrind`, ensuring consistent compilation and testing.
- Memory Management: Executed rigorous memory profiling using Valgrind within the containerized environment (`docker-compose run --rm campus-guard make valgrind`).
- Verification Results: The system completed execution with zero memory leaks and zero errors. The Valgrind heap summary confirmed flawless dynamic memory management across all pattern implementations, with all 56 heap allocations cleanly matched by 56 frees (0 bytes in use at exit).

Contributions:

Jordan: Most of my work was on designing and creating the final UML, I then was responsible for the code for the Adapter and Strategy design patterns. I also made the `main.cpp` file and the user stories behind them.

Nasiha: I was mainly responsible for implementing the Mediator and Command design patterns. I also conducted valgrind testing as well as a GDB investigation on an existing bug. Furthermore, I designed one of the sequence diagrams and the third own-choice diagram, a state machine diagram.

Lwandiso: I implemented the State and Facade design patterns and was responsible for the final system integration, environment configuration, and Task 6. I containerized the application using Docker and Docker Compose, authored the configuration for a reproducible C++ build environment, and executed the final rigorous Valgrind profiling within the container to verify memory management. This ensured the integrated patterns worked together flawlessly and the final build executed with zero memory leaks or errors.